

Prise en main du Framework Alpine.js

Qu'est-ce que le Framework Alpine.js ?

AlpineJS est un micro-Framework JavaScript léger (3 Ko gzippé) pour réaliser des interactions de base dans des pages web. Le code s'écrit à l'aide de directives que l'on ajoute aux balises HTML et dont la syntaxe s'inspire fortement de celle du Framework Vue.js, mais en beaucoup plus simple et sans build system.

Présentation

Alpine.JS est un Framework JavaScript développé par [Caleb Porzio](#) permettant de manipuler le DOM et de dynamiser vos pages web. Pour concevoir Alpine.JS, Caleb Porzio s'est inspiré de la syntaxe VueJS et du Framework CSS TailwindCSS. Le Framework met à disposition des directives et des propriétés que l'on utilise directement dans les balises HTML.

C'est un Framework léger (6,4kb GZippé et minifié) qui s'utilise simplement via un lien CDN ou une installation via npm. Pas de DOM virtuel ni d'étape de build.

Alpine.JS vient se placer entre le VanillaJS/Jquery et les Frameworks comme React/VueJS, il permet de très facilement manipuler le DOM pour par exemple :

- Masquer/montrer des nœuds du DOM selon certaines conditions
- Manipuler dynamiquement des classes
- Modifier l'interface selon les actions de l'utilisateur
- Lier des données saisies par l'utilisateur à d'autres éléments

AlpineJS est idéal si vous souhaitez développer des composants dynamiques comme des onglets, des modals, des dropdown menu, des sidebares escamotables etc ... dans un projet de type "server-side rendering (Laravel, CMS, Django, Ruby on Rails ...) Ce n'est pas, en revanche, un Framework adapté pour créer des applications de type single page avec un système de routes pour naviguer dans l'application.

Il peut être utilisé sans avoir besoin d'utiliser/maintenir un fichier js externe ou du JavaScript dans une balise `<script>`. Cependant, selon les cas, cela reste possible et le code peut être organisé en plusieurs fichiers.

Installation

Depuis un CDN via la balise `<script>`

Il suffit d'ajouter simplement la ligne suivante dans la section `<head>` ou juste avant la fin de la section `</body>` :

```
<script defer src="https://unpkg.com/alpinejs@3.x.x/dist/cdn.min.js"></script>
```

⚠ Remplace 3.x.x par la dernière version stable ou celle souhaitée.

A partir de NPM

Installer le package Alpine.js depuis npm

Npm i alpinejs

Puis inclure AlpineJS dans vos scripts

Import 'alpinejs'

Les directives

Les directives sont des attributs placés dans les balises html et spécifiques au Framework. Elles servent à transformer ou modifier le DOM (Document Object Model).

x-data

Cette directive indique au Framework d'initialiser un nouveau composant avec des données au format objet. Le champ d'application de ces données s'applique au niveau du composant dans lequel x-data est déclaré.

```
// ici le champ d'application s'applique au niveau du <div></div> et donc aussi
aux nœuds imbriqués dans ce <div>
<div x-data="{ foo: 'bar' }">
  <h1>Hello word!</h1>
</div>
```

Les datas peuvent être transmises directement, via un objet, dans la directive x-data ou indirectement à l'aide d'une fonction définie soit dans une balise <script> soit dans un fichier .js externe :

```
<div x-data="helloWord()">
  <button @click="display()">Display message</button>
  <div>Message: <span x-text="messageDisplay"></span></div>
</div>
```

```
<script>
function helloWord() {
  return {
    message: 'Hello Word!',
    messageDisplay: '',
    display() {
      this.messageDisplay = this.message;
    }
  }
}
</script>
```

// quand on clique sur le bouton "Display message" le message "Hello Word!" est affiché dans la balise

Les datas peuvent être transmises directement, via un objet, dans la directive `x-data` ou indirectement à l'aide d'une fonction définie soit dans une balise `<script>` soit dans un fichier `.js` externe :

```
<div x-data="helloWord()">
  <button @click="display()">Display message</button>
  <div>Message: <span x-text="messageDisplay"></span></div>
</div>

<script>
function helloWord() {
  return {
    message: 'Hello Word!',
    messageDisplay: '',
    display() {
      this.messageDisplay = this.message;
    }
  }
}
</script>
```

// quand on clique sur le bouton "Display message" le message "Hello Word!" est affiché dans la balise `<span>`

On notera que depuis la méthode `display()`, Alpine.js nous permet d'accéder aux autres propriétés de l'objet via le mot clé `this`

Il est aussi possible de récupérer des données d'une API et de les transmettre à la directive `x-data`. Pour cela, nous assignerons une fonction à la directive `x-data` et cette fonction utilisera l'API `fetch` pour récupérer les données externes.

```
<div x-data="pokeSearch()">
  <!-- rest of HTML -->
  <button
    type="submit"
    @click="fetchPokemon()"
    :disabled="isLoading"
  >
    Search
  </button>
  <script>
function pokeSearch() {
  return {
    // other default properties
    isLoading: false,
    pokemon: null,
    fetchPokemon() {
      this.isLoading = true;
      fetch(`https://pokeapi.co/api/v2/pokemon/${this.pokemonSearch}`)
        .then(res => res.json())
        .then(data => {
          this.isLoading = false;
          this.pokemon = data;
        });
    }
  }
}
  </script>
</div>
```

x-init

La directive x-init permet d'exécuter une expression JavaScript au moment de l'initialisation du composant.

```
// initialisation de la propriété "message"
<div x-data="{ message: '' }" x-init="message = 'Hello buddy!'"></div>
```

```
// initialisation de la propriété "message" à partir de données externes (API)
<div x-data="{ message: '' }" x-init="message = fetch('https://api.kanye.rest')
  .then(response => response.json()) .then(data => quote = data.quote)"></div>
```

```
// utilisation d'une fonction
<div x-data="{ message: '' }" x-init="message = displayQuote()"></div>
<script>
function displayQuote() {
  return {
    fetch('https://api.kanye.rest') .then(response => response.json()) .then(data =>
quote = data.quote);
  }
}
</script>
```

Si vous souhaitez exécuter l'expression JavaScript **après** que Alpine.js ait effectué ses premières mises à jour du DOM, vous pouvez retourner une fonction de callback sur x-init, et elle sera exécutée après.

```
<div x-data="{ message: '' }" x-init="return () => { // nous avons accès à l'état
post mise à jour du DOM ici ... // }"></div>
```

x-show

Cette directive ajoute ou retire un style `display:none` sur l'élément ciblé selon que l'expression renvoie `true` ou `false`. Cette directive permet donc de masquer ou montrer un élément selon la valeur booléenne renvoyée par l'expression.

x-show met à notre disposition les transitions CSS pour appliquer des effets plus élaborés.

```
<div x-show.transition="open">
  These contents will be transitioned in and out.
</div>
```

Liste des transitions disponibles :

Block error: "Array to string conversion" in block type: "table"

Ces transitions peuvent être utilisées en conjonction les unes avec les autres :

x-
show.transition.in.duration.100ms.origin.top.right.opacity.scale.85.out.duration.200ms.origin.bottom
.left.opacity.scale.95

x-show attend que les enfants aient terminé leur transition. Si vous voulez contourner ce comportement, ajoutez le modificateur `.immediate`

```
<div x-show.immediate="open">
  <div x-show.transition="open">
  </div>
```

x-bind

Cette directive lie la valeur d'un attribut HTML au résultat d'une expression JavaScript. L'expression JavaScript a accès à toutes les clés de l'objet de données du composant et la valeur de l'attribut est mis à jour chaque fois que les données de l'objet sont modifiées.

```
// structure
<input x-bind:[attribute]="[expression]">
```

```
// écriture longue
<input x-bind:type="inputType">
```

```
// écriture courte
<input :type="inputType">
```

x-bind pour les attributs de classe

Pour les classes, on passe un objet dont la clé est le nom de la classe et la valeur, une expression boolean qui détermine si cette classe est appliquée ou pas à l'élément.

```
<div x-bind:class="{ 'hidden': foo }"></div>
```

// la classe "hidden" ne sera appliquée que lorsque "foo" aura la valeur "true"

x-bind pour les attributs de type boolean

x-bind supporte les [attributs booléens](#) de la même manière que les attributs ayant une valeur, utilisant une variable ou une expression javascript retournant true or false.

```
<button x-bind:disabled="myVar">Click me</button>
```

// l'attribut "disabled" sera ajouté ou retiré à la balise "button" selon que "myVar" est true ou false respectivement.

Les [attributs booléens](#) sont pris en charge conformément à la spécification HTML

x-on

Cette directive attache un "event listener" à l'élément auquel elle est rattachée. Lorsque cet événement est émis, l'expression JavaScript définie comme sa valeur est exécutée.

Si une donnée est modifiée dans l'expression, les attributs des autres éléments "liés" à cette donnée seront mis à jour.

```
// structure
<button x-on:[event]="[expression]"></button>
```

```
// écriture longue
<button x-on:click="foo = 'bar'"></button>
```

```
// écriture courte
<button @:click="foo = 'bar'"></button>
```

keydown modifiers

Il est possible de spécifier des touches à écouter en utilisant les modificateurs de touches annexés à la directive x-on:keydown. Notez que les modificateurs sont des versions kebab des valeurs de Event.key. enter, escape, arrow-up, arrow-down

```
<input type="text" x-on:keydown.escape="open = false">
```

Les combinaisons de touches peuvent également être écouté : x-on:keydown.ctl.enter="foo"

.away modifier

Lorsque le modificateur .away est présent, le gestionnaire d'événement ne sera exécuté que si l'événement provient d'une source autre que lui-même ou ses enfants.

Ceci est utile pour masquer les dropdowns et les modals lorsqu'un utilisateur clique à côté de ces éléments.

```
<div x-on:click.away="showModal = false"></div>
```

.prevent modifier

L'ajout de `.prevent` à l'écoute d'un événement appelle `preventDefault` sur l'événement déclenché.

```
<input type="checkbox" x-on:click.prevent>
```

Dans l'exemple ci-dessus, cela signifie que la case à cocher ne sera pas réellement cochée lorsqu'un utilisateur cliquera dessus.

.stop modifier

L'ajout de `.stop` à l'écoute d'un événement appelle `stopPropagation` sur l'événement déclenché.

```
<div x-on:click="foo = 'bar'"><button x-on:click.stop></button></div>
```

Dans l'exemple ci-dessus, cela signifie que l'événement "click" ne s'étendra pas du bouton jusqu'au `<div>` extérieur. En d'autres termes, lorsqu'un utilisateur clique sur le bouton, `foo` ne sera pas réglé sur "bar".

.window modifier

L'ajout de `.window` à un listener d'événement, installera le listener sur l'objet global `window` au lieu du nœud DOM sur lequel il est déclaré. Ceci est utile lorsque vous souhaitez modifier l'état d'un composant lorsque quelque chose change avec la fenêtre, comme son redimensionnement.

```
<div x-on:resize.window="isOpen = window.outerWidth > 768 ? false : open"></div>
```

Dans l'exemple ci-dessus, lorsque la fenêtre s'agrandit de plus de 768 pixels de large, nous fermons le modal/dropdown, sinon nous maintenons le même état.

Vous pouvez également utiliser le modificateur `.document` à la place de `.window`

.once modifier

L'ajout du modificateur `.once` à un listener d'événement garantira que le listener ne sera traité qu'une seule fois. Cela est utile pour les choses que vous ne voulez faire qu'une seule fois.

```
<button x-on:mouseenter.once="fetchSomething()"></button>
```

.debounce modifier

Le modificateur `.debounce` vous permet de "débloquer" un gestionnaire d'événements. En d'autres termes, le gestionnaire d'événements ne fonctionnera PAS tant qu'un certain temps ne se sera pas écoulé depuis le dernier événement qui s'est déclenché. Lorsque le gestionnaire est prêt à être appelé, le dernier appel du gestionnaire sera exécuté.

```
<input x-on:input.debounce="fetchSomething()">
```

Le temps d'attente par défaut pour le déblocage est de 250 millisecondes.

Si vous souhaitez personnaliser ce temps, vous pouvez spécifier un temps d'attente personnalisé de la sorte :

```
<input x-on:input.debounce.750="fetchSomething()">
<input x-on:input.debounce.750ms="fetchSomething()">
```

x-model

La directive x-model ajoute une "liaison de données bidirectionnelle" à un élément. En d'autres termes, la valeur de l'élément d'entrée sera maintenue en synchronisation avec la valeur de l'élément du composant

```
// structure
<input type="text" x-model="[data item]">

// exemple
<input type="text" x-model="foo">
```

x-model détecte les changements sur les champs inputs, les cases à cocher, les boutons radio, les textareas, les menus de sélections et les menus de sélections multiples.

.debounce modifier

Le modificateur debounce permet d'ajouter un "débloqueur" à la mise à jour d'une valeur. En d'autres termes, le gestionnaire d'événements ne s'exécutera PAS avant qu'un certain temps ne se soit écoulé depuis le dernier événement qui a déclenché la mise à jour. Lorsque le gestionnaire est prêt à être appelé, le dernier appel du gestionnaire sera exécuté.

Le temps de déblocage par défaut est de 250 millisecondes.

Vous pouvez personnaliser ce temps d'attente de la façon suivante :

```
<input x-model.debounce.750="search">
<input x-model.debounce.750ms="search">
```

x-text

La directive x-text fonctionne de la même manière que la directive x-bind, sauf qu'au lieu de mettre à jour la valeur d'un attribut, elle met à jour le innerText d'un élément.

```
// structure
<span x-text="[expression]"

// exemple
<span x-text="foo"></span>
```

x-html

La directive x-html fonctionne de la même manière que la directive x-bind, sauf qu'au lieu de mettre à jour la valeur d'un attribut, elle met à jour le innerHTML d'un élément.

```
// structure
<span x-html="[expression]"

// exemple
<span x-html="foo"></span>
```

x-ref

La directive x-ref offre un moyen pratique de récupérer des éléments DOM bruts de votre composant. En définissant un attribut x-ref sur un élément, vous le mettez à la disposition de tous les gestionnaires d'événements à l'intérieur d'un objet appelé \$refs.

C'est une alternative utile à la définition d'ids et à l'utilisation de document.querySelector.

```
// structure
<div x-ref="[ref name]"></div><button x-on:click="$refs.[ref name].innerText = 'bar'"></button>

// exemple
<div x-ref="foo"></div><button x-on:click="$refs.foo.innerText = 'bar'"></button>
```

Vous pouvez également lier des valeurs dynamiques pour x-ref si besoin.

```
<span :x-ref="item.id"></span>
```

x-if

Pour les cas où la directive x-show n'est pas suffisante (x-show définit un élément display: none aucun s'il est faux), x-if peut être utilisé pour retirer effectivement un élément complètement du DOM.

Il est important que la directive x-if soit utilisée sur une balise <template></template> car AlpineJS n'utilise pas de DOM virtuel.

```
// structure
<template x-if="[expression]"><div>Some Element</div></template>

// exemple
<template x-if="true"><div>Some Element</div></template>
```

La directive x-if doit avoir un élément racine unique (ici <div>) à l'intérieur de la balise <template></template>

x-for

La directive x-for est disponible dans les cas où vous souhaitez créer de nouveaux nœuds DOM pour chaque élément d'un tableau.

```
// exemple
<template x-for="item in items" :key="item">
  <div x-text="item"></div>
</template>
```

La clé :key est optionnelle mais HAUTEMENT recommandée.

Si vous voulez accéder à l'index actuel de l'itération, utilisez la syntaxe suivante :

```
<template x-for="(item, index) in items" :key="index">
  <!-- You can also reference "index" inside the iteration if you need. -->
  <div x-text="index"></div>
</template>
```

Il est possible d'imbriquer des directives x-for mais vous DEVEZ "envelopper" chaque boucle dans un élément (ici un élément <div>)

```
<template x-for="item in items">
  <div>
    <template x-for="subItem in item.subItems">
      <div x-text="subItem"></div>
    </template>
  </div>
</template>
```

x-transition

AlpineJS propose 6 directives différentes pour gérer la transition entre les états "cachés" et "montrés" d'un élément. Ces directives fonctionnent à la fois avec la directive x-show ET la directive x-if.

Block error: "Array to string conversion" in block type: "table"

```
<div
  x-show="open"
  x-transition:enter="transition ease-out duration-300"
  x-transition:enter-start="opacity-0 transform scale-90"
  x-transition:enter-end="opacity-100 transform scale-100"
  x-transition:leave="transition ease-in duration-300"
  x-transition:leave-start="opacity-100 transform scale-100"
  x-transition:leave-end="opacity-0 transform scale-90"
>...</div>
<template x-if="open">
  <div
    x-transition:enter="transition ease-out duration-300"
    x-transition:enter-start="opacity-0 transform scale-90"
    x-transition:enter-end="opacity-100 transform scale-100"
    x-transition:leave="transition ease-in duration-300"
    x-transition:leave-start="opacity-100 transform scale-100"
    x-transition:leave-end="opacity-0 transform scale-90"
  >...</div>
```

```
</template>
```

x-cloak

Les attributs x-cloak sont retirés des éléments lorsque AlpineJS s'initialise. Ceci est utile pour cacher les DOM pré-initialisés. Typiquement utilisé pour ajouter le style global suivant :

```
<style>
  [x-cloak] { display: none; }
</style>
```

Les propriétés magiques

\$el

\$el est une propriété magique qui peut être utilisée pour récupérer le nœud DOM du composant racine.

```
// exemple
<div x-data>
  <button @click="$el.innerHTML = 'foo'">Replace me with "foo"</button>
</div>
```

\$refs

\$refs est une propriété magique qui peut être utilisée pour récupérer les éléments DOM marqués d'un x-ref à l'intérieur du composant. Ceci est utile lorsque vous devez manipuler manuellement des éléments DOM.

```
// exemple
<span x-ref="foo"></span>
<button x-on:click="$refs.foo.innerText = 'bar'"></button>
```

\$event

\$event est une propriété magique qui peut être utilisée dans un auditeur d'événement pour récupérer l'objet "Event" du navigateur natif.

```
// exemple
<input x-on:input="alert($event.target.value)">
```

\$dispatch

\$dispatch est un raccourci pour créer un CustomEvent et l'expédier en utilisant .dispatchEvent() en interne. Il existe de nombreux cas d'utilisation pour la transmission de données entre les composants et autour de ceux-ci à l'aide d'événements personnalisés. Lisez ici pour plus d'informations sur le système CustomEvent sous-jacent dans les navigateurs.

```
// exemple
<div @custom-event="console.log($event.detail.foo)">
  <button @click="$dispatch('custom-event', { foo: 'bar' })">
    <!-- When clicked, will console.log "bar" -->
  </button>
</div>
```

Vous remarquerez que toute donnée passée comme deuxième paramètre à \$dispatch('some-event', { some : 'data' }), devient disponible grâce à la nouvelle propriété "detail" des événements : \$event.detail.some. L'ajout de données d'événements personnalisés à la propriété .detail est une pratique courante pour les CustomEvents dans les navigateurs. Pour plus d'informations, [cliquez ici](#).

Vous pouvez également utiliser \$dispatch() pour déclencher des mises à jour de données pour les directives de binding x-model.

```
// exemple
<div x-data="{ foo: 'bar' }">
  <span x-model="foo">
    <button @click="$dispatch('input', 'baz')">
      <!-- After the button is clicked, `x-model` will catch the bubbling
"input" event, and update foo to "baz". -->
    </button>
  </span>
</div>
```

\$nextTick

\$nextTick est une propriété magique qui vous permet de n'exécuter une expression donnée qu'APRÈS que AlpineJS ait fait ses mises à jour réactives de DOM. Ceci est utile pour les fois où vous voulez interagir avec l'état DOM APRÈS qu'il ait reflété les mises à jour de données que vous avez faites.

```
// exemple
<div x-data="{ fruit: 'apple' }">
  <button
    x-on:click="
      fruit = 'pear';
      $nextTick(() => { console.log($event.target.innerText) });
    "
    x-text="fruit"
  >
  </button>
</div>
```

\$watch

Vous pouvez "surveiller" la propriété d'un composant avec la méthode magique \$watch. Dans l'exemple ci-dessus, lorsque l'on clique sur le bouton et qu'on le modifie, le callback fourni se déclenche et console.log la nouvelle valeur.

```
// exemple
<div x-data="{ open: false }" x-init="$watch('open', value =>
console.log(value))">
  <button @click="open = ! open">Toggle Open</button>
</div>
```

Les concepts de base Alpine.js

Fonctionnalité	Description	Exemple
x-data	Déclare des données locales	x-data="{ open: false }"
x-show	Affiche ou cache un élément	x-show="open"
x-bind ou :	Lie un attribut HTML à une donnée	:class="active ? 'on' : 'off'"
x-on ou @	Gère un événement	@click="toggle()"
x-text	Met du texte dans l'élément	x-text="message"
x-model	Liaison bidirectionnelle (input)	x-model="name"

Exemples de base

Exemple 1 : Création d'une modal

Dans cet exemple, nous allons créer un bouton pour ouvrir la modal, celle-ci étant masquée par défaut. Dans la modal, un autre bouton sera créé pour la fermer.

Initialisation du composant principal avec un objet

Nous commençons par initialiser un composant dans une balise `<div>` avec un objet à l'aide de la directive `x-data`. L'objet possède une propriété que nous appelons `modalOpen` et nous initialisons sa valeur à `false`. La propriété `modalOpen` aura sa valeur à `false` lorsque la modale sera fermée et à `true` lorsqu'elle sera ouverte. Par défaut, la modal est fermée, c'est pourquoi nous initialisons `modalOpen` à `false`.

```
<div
  x-data="{ modalOpen: false }"
>
...
</div>
```

Création du bouton d'ouverture de la modal

Sur ce bouton, nous ajoutons une directive `x-on` pour attacher un event-listener "click". Nous utilisons ici la notation courte `@click`. Lorsque l'utilisateur clique sur ce bouton d'ouverture, nous passons la valeur de la propriété `modalOpen` à `true`.

Nous utilisons également sur cet event-listener la propriété magique `$nextTick` pour exécuter une fonction fléchée après la mise à jour des données. Cette fonction donne le focus sur l'élément DOM `modalCloseButton` identifiée ainsi à l'aide d'une directive `x-ref`. Cet élément n'est autre que le bouton de fermeture de la modal.

```
<div
  x-data="{ modalOpen: false }"
>
<button class="bg-blue-700 text-white px-4 py-3 mt-4 text-sm rounded"
  @click="modalOpen = true
  $nextTick(() => $refs.modalCloseButton.focus())"
>
Open this modal
</button>
...
</div>
```

Création de la modal et de son bouton de fermeture

La modal est positionnée en absolute via des classes CSS.

La balise <div> qui définit la modal contient une directive x-show qui masque la modal lorsque la valeur de la propriété modalOpen est à false ou la montre lorsqu'elle est à true.

Le bouton de fermeture contient une directive x-on qui écoute sur le clic de l'utilisateur. Nous utilisons ici la notation courte @click Lorsque l'utilisateur clique sur le bouton de fermeture, la valeur de la propriété modalOpen est passé à false La directive x-show branchée sur la valeur de propriété modalOpen reçoit alors le signal de masquer la modal.

```
<div
    class="mx-auto absolute top-0 left-0 w-full h-full flex items-
center shadow-lg overflow-y-auto"
    x-show="modalOpen"
>
    <div class="bg-gray-400 container mx-auto rounded p-4 mt-2
overflow-y-auto">
        <div class="bg-white rounded px-8 py-8">
            <h1 class="font-bold text-2xl mb-3">Modal</h1>
            <div class="modal-body">
                <p>I am open, isn't it!
                </p>
            </div>
            <div class="mt-4">
                <button
                    class="bg-blue-700 text-white px-4 py-3 mt-4 text-
sm rounded"
                    @click="modalOpen = false"
                    x-ref="modalCloseButton"
                >
                    Close this modal
                </button>
            </div>
        </div>
    </div>
</div>
```

Code complet :

Copiez/collez ce code dans un fichier `modal.html` puis ouvrez le dans une fenêtre de votre navigateur.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>AlpineJS - Modal</title>

  <!-- Alpine.js v3 (dernière version via CDN) -->
  <script src="https://cdn.jsdelivr.net/npm/alpinejs@3.x.x/dist/cdn.min.js"
defer></script>

<style>
  body {
    background-color: #e5e7eb; /* Equivalent to Tailwind's bg-gray-200 */
    font-family: sans-serif;
    margin: 0;
    padding: 0;
  }

  .container {
    max-width: 960px;
    margin: 3rem auto;
    padding: 0 1rem;
  }

  h2 {
    font-size: 1.5rem;
    font-weight: bold;
  }

  .btn {
    background-color: #1d4ed8; /* Tailwind's blue-700 */
    color: white;
    padding: 0.75rem 1rem;
    font-size: 0.875rem;
    border: none;
    border-radius: 0.375rem;
    cursor: pointer;
    margin-top: 1rem;
  }

  .modal-overlay {
    position: fixed;
    top: 0; left: 0;
    width: 100%;
    height: 100%;
    background-color: rgba(0,0,0,0.3);
    display: flex;
    align-items: center;
    justify-content: center;
    z-index: 1000;
  }

  .modal-container {
```

```

    background-color: #9ca3af; /* Tailwind's gray-400 */
    border-radius: 0.5rem;
    padding: 1rem;
    width: 100%;
    max-width: 640px;
}

.modal-content {
    background-color: white;
    border-radius: 0.5rem;
    padding: 2rem;
}

.modal-content h1 {
    font-size: 1.5rem;
    font-weight: bold;
    margin-bottom: 1rem;
}

.modal-body p {
    margin: 0;
}

.mt-4 {
    margin-top: 1rem;
}

[x-cloak] {
    display: none !important;
}
</style>
</head>

<body>
  <div class="container">
    <h2>AlpineJS - Modal</h2>

    <div x-data="{ modalOpen: false }">
      <button
        class="btn"
        @click="modalOpen = true; $nextTick(() => $refs.modalCloseButton.focus())"
      >
        Ouverture d'une fenêtre modale
      </button>

      <!-- Modal -->
      <div
        class="modal-overlay"
        x-show="modalOpen"
        x-transition
        @keydown.escape.window="modalOpen = false"
        x-cloak
      >
        <div class="modal-container">
          <div class="modal-content">
            <h1>Modal</h1>
            <div class="modal-body">
              <p>Je suis ouverte, n'est-ce pas !</p>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
</body>

```

```
<div class="mt-4">
  <button
    class="btn"
    @click="modalOpen = false"
    x-ref="modalCloseButton"
  >
    Fermeture d'une fenêtre modale
  </button>
</div>
</div>
</div>
</div>
<!-- End Modal -->
</div>
</div>
</body>
</html>
```

Exemple 2 : Création d'une sidebar escamotable

Dans cet exemple, une icône représentant un menu hamburger permet d'ouvrir une sidebar contenant un menu d'items. Cette sidebar dispose d'une icône "X" pour la masquer. Cet exemple dans son fonctionnement est assez proche de celui de la modal. Un élément est masqué par défaut et positionné en "absolute". Un clic de l'utilisateur à un endroit de l'interface ouvre cet élément, un clic à un autre endroit ferme l'élément.

Initialisation du composant principal avec un objet

Le composant principal est une balise <nav> dans laquelle on initialise un objet contenant une propriété sidebarOpen dont la valeur est à false. Cette propriété nous servira à indiquer si la sidebar est masquée (valeur à false) ou pas (valeur à true).

```
<nav
  class="container mx-auto px-4 flex items-center justify-between py-4"
  x-data="{ sidebarOpen: false }"
>
...
</nav>
```

Création du bouton d'ouverture de la sidebar

Une icône hamburger permettra à l'utilisateur d'ouvrir la sidebar à l'aide d'une directive x-on qui écoute sur l'événement clique. Celle-ci est écrite dans sa version courte @click et modifie à true la valeur de la propriété sidebarOpen lorsque l'utilisateur clique sur l'icône.

```
<button
  @click="sidebarOpen = true"
>
<svg fill="currentColor" viewBox="0 0 24 24" width="24" height="24">
<path class="heroicon-ui" d="M4 5h16a1 1 0 010 2H4a1 1 0 110-2zm0 6h16a1 1 0 010 2H4a1 1 0 110-2zm0 6h16a1 1 0 010 2H4a1 1 0 110-2z"/>
</svg>
</button>
```

Création de l'élément sidebar

L'élément sidebar est définie dans une balise <div> positionnée de façon absolue via des classes TailwindCSS. Une directive x-show masque ou montre la sidebar selon la valeur de la propriété sidebarOpen

Une directive x-on (ici en notation courte @click) écoute un événement "clique" en dehors de l'élément à l'aide du "modifier" .away Ainsi lorsque l'utilisateur clique en dehors de l'élément sidebar, la valeur de la propriété sidebarOpen est passé à false et la directive x-show masque l'élément sidebar.

```
<div
  class="absolute top-0 left-0 bg-white border border-right h-full w-48"
  x-show="sidebarOpen"
  @click.away="sidebarOpen = false"
>
...
</div>
```

Code complet :

Copiez/collez ce code dans un fichier sidebar.html puis ouvrez le dans une fenêtre de votre navigateur.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>AlpineJS - Sidebar</title>

  <!-- Alpine.js v3 -->
  <script src="https://cdn.jsdelivr.net/npm/alpinejs@3.x.x/dist/cdn.min.js"
defer></script>

<style>
  body {
    margin: 0;
    font-family: sans-serif;
    background-color: #d1d5db; /* bg-gray-300 */
    color: #1f2937; /* text-gray-800 */
  }

  header {
    border-top: 4px solid #1d4ed8; /* border-blue-700 */
    background-color: white;
    box-shadow: 0 2px 4px rgba(0,0,0,0.1); /* shadow-md */
  }

  nav.container {
    max-width: 960px;
    margin: 0 auto;
    padding: 1rem;
    display: flex;
    align-items: center;
    justify-content: space-between;
  }

  .sidebar {
    position: fixed;
    top: 0;
    left: 0;
    background-color: white;
    border-right: 1px solid #ccc;
    height: 100%;
    width: 12rem; /* 48px * 4 = 192px */
    z-index: 1000;
    padding-top: 1rem;
  }

  .sidebar button.close-btn {
    font-size: 1.5rem;
    margin-right: 0.75rem;
    margin-top: 0.25rem;
    background: none;
    border: none;
    cursor: pointer;
  }
```

```

.sidebar ul {
  list-style: none;
  padding: 0;
  margin-top: 1rem;
}

.sidebar a {
  display: flex;
  align-items: center;
  padding: 0.75rem 1rem;
  text-decoration: none;
  color: inherit;
  transition: background-color 0.2s ease;
}

.sidebar a:hover {
  background-color: #e5e7eb; /* hover:bg-gray-200 */
}

.sidebar svg {
  margin-right: 0.5rem;
  fill: #4b5563; /* text-gray-600 */
}

.menu-button {
  background: none;
  border: none;
  cursor: pointer;
  padding: 0.5rem;
}

[x-cloak] {
  display: none !important;
}
</style>
</head>

<body>
  <header>
    <nav class="container" x-data="{ sidebarOpen: false }">
      <div>
        <button class="menu-button" @click="sidebarOpen = true" aria-label="Open sidebar">
          <svg fill="currentColor" viewBox="0 0 24 24" width="24" height="24">
            <path d="M4 5h16a1 1 0 0 1 0 2H4a1 1 0 0 1 0 2zm0 6h16a1 1 0 0 1 0 2H4a1 1 0 0 1 0 2zm0 6h16a1 1 0 0 1 0 2H4a1 1 0 0 1 0 2z"/>
          </svg>
        </button>
      </div>

      <!-- Sidebar -->
      <div
        class="sidebar"
        x-show="sidebarOpen"
        x-cloak
        @click.away="sidebarOpen = false"
        x-transition
      >

```

```

        <div class="flex justify-end">
            <button class="close-btn" @click="sidebarOpen = false" aria-label="Close
sidebar">
                x
            </button>
        </div>
        <ul>
            <li>
                <a href="#">
                    <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
width="24" height="24">
                        <path d="M12 12a5 5 0 1 1 0-10 5 5 0 0 1 0 10zm0-2a3 3 0 1 0 0-6 3
3 0 0 0 0 6zm9 11a1 1 0 0 1-2 0v-2a3 3 0 0 0-3-3H8a3 3 0 0 0-3 3v2a1 1 0 0 1-2 0v-
2a5 5 0 0 1 5-5h8a5 5 0 0 1 5 5v2z"></path>
                    </svg>
                    <span>Account</span>
                </a>
            </li>
            <li>
                <a href="#">
                    <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
width="24" height="24">
                        <path d="M9 4.58V4c0-1.1.9-2 2-2h2a2 2 0 0 1 2 2v.58a8 8 0 0 1
1.92 1.11l.5-.29a2 2 0 0 1 2.74.73l1 1.74a2 2 0 0 1-.73 2.73l-.5-.29a8.06 8.06 0 0
1 0 2.22l.5.3a2 2 0 0 1 .73 2.72l-1 1.74a2 2 0 0 1-2.73.73l-.5-.3A8 8 0 0 1 15
19.43V20a2 2 0 0 1-2 2h-2a2 2 0 0 1-2-2v-.58a8 8 0 0 1-1.92-1.11l-.5-.29a2 2 0 0 1-
2.74-.73l-1-1.74a2 2 0 0 1 .73-2.73l.5-.29a8.06 8.06 0 0 1 0-2.22l-.5-.3a2 2 0 0
1-.73-2.72l1-1.74a2 2 0 0 1 2.73-.73l.5.3A8 8 0 0 1 9 4.57zM7.88 7.64l-.54.51-
1.77-1.02-1 1.74 1.76 1.01-.17.73a6.02 6.02 0 0 0 2.78l.17.73-1.76 1.01 1 1.74
1.77-1.02.54.51a6 6 0 0 0 2.4 1.41l.72.2V20h2v-.204l.71-.2a6 6 0 0 0 2.41-1.41.54-
.51 1.77 1.02 1-1.74-1.76-1.01.17-.73a6.02 6.02 0 0 0 2.78l-.17-.73 1.76-1.01-1-
1.74-1.77 1.02-.54-.51a6 6 0 0 0-2.4-1.41-.72-.2V4h-2v2.04l-.71.2a6 6 0 0 0-2.41
1.4zM12 16a4 4 0 1 1 0-8 4 4 0 0 1 0 8zm0-2a2 2 0 1 0 0-4 2 2 0 0 0 0 4z"></path>
                    </svg>
                    <span>Settings</span>
                </a>
            </li>
            <li>
                <a href="#">
                    <svg xmlns="http://www.w3.org/2000/svg" width="24" height="24"
viewBox="0 0 24 24">
                        <path d="M0 0h24v24H0z" fill="none"></path>
                        <path d="M10.09 15.59L11.5 17l5-5-5-1.41 1.41L12.67
11H3v2h9.67l-2.58 2.59zM19 3H5c-1.11 0-2 .9-2 2v4h2V5h14v14H5v-4H3v4c0 1.1.89 2 2
2h14c1.1 0 2-.9 2-2V5c0-1.1-.9-2-2-2z"></path>
                    </svg>
                    <span>Logout</span>
                </a>
            </li>
        </ul>
    </div>
    <!-- End Sidebar -->
</nav>
</header>
</body>
</html>

```

Exemple 3 : Création d'onglets

Dans ce dernier exemple commenté, nous allons construire une liste `<ul><li>`, chaque item `<li>` représentant un onglet. Le contenu lié à l'onglet est inséré dans une balise `<div>` en dehors de la liste. Par défaut c'est l'onglet 1 qui est montré et actif. Lorsque l'utilisateur clique sur un des deux autres onglets, son contenu remplace le contenu de l'onglet précédent et l'onglet cliqué devient actif.

Initialisation du composant principal avec un objet

Le composant principal est une balise `<div>` contenant une directive `x-data` qui initialise un objet comportant une propriété `tab` initialisé avec la valeur `tab1`

```
<div
  class="container mx-auto p-4 mb-4"
  x-data="{ tab: 'tab1' }"
>
```

Onglet actif et inactif

L'onglet est représenté par une balise `<li>` qui contient deux directives.

Une directive `x-bind` lié à un attribut de class (noté avec sa version courte ici `:class`). Cette directive permet d'appliquer des classes à l'élément selon la valeur booléenne renvoyée par une expression JavaScript. Ici, dans l'onglet 1, cette expression teste si la propriété `tab` a la valeur `tab1`, si oui, les classes de la directive sont appliquées à l'élément pour visuellement montrer que l'onglet est actif (fond blanc, bordure, texte en bleu ...).

Une seconde directive `x-on` écoute l'évènement `click` de l'utilisateur (en notation courte ici: `@click`). Si l'onglet est cliqué, la valeur de la propriété `tab` est passé à `tab1`

```
<li class="-mb-px mr-1">
  <a class="inline-block rounded-t py-2 px-4 font-semibold hover:text-blue-800"
    :class="{ 'bg-white text-blue-700 border-1 border-t border-r' : tab === 'tab1'
  }" href="#"
    @click.prevent="tab = 'tab1'"
  >Tab 1</a>
</li>
```

Contenu de l'onglet, visible ou masqué

Le contenu des onglets est inséré dans une balise `<div>` juste en dessous de la liste `<ul><li>` contenant les onglets. Leur apparition ou leur masquage est déterminé via une directive `x-show` qui évalue la valeur booléenne de l'expression `tab === 'tab1'`. Si l'expression renvoie `true`, le contenu de l'élément `<div>` est montré.

```
<div
  x-show="tab === 'tab1'"
>
  <strong>Tab I content.</strong> Lorem ipsum dolor sit amet, consectetur
adipiscing elit. Nesciunt sunt, consectetur eos
  quod perferendis mollitia consequuntur natus, porro molestiae qui iusto deserunt
rerum tempore
  voluptatum itaque. Ad, nisi esse cum quidem consequuntur ullam obcaecati.
</div>
```

Code complet :

Copiez/collez ce code dans un fichier `tabs.html` puis ouvrez le dans une fenêtre de votre navigateur.

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>AlpineJS - Tabs</title>

  <!-- Alpine.js v3 (dernier CDN) -->
  <script src="https://cdn.jsdelivr.net/npm/alpinejs@3.x.x/dist/cdn.min.js"
defer></script>

  <style>
    body {
      background-color: #e5e7eb; /* bg-gray-200 */
      color: #1f2937; /* text-gray-800 */
      font-family: sans-serif;
      margin: 0;
      padding: 0;
    }

    .container {
      max-width: 960px;
      margin: 2rem auto;
      padding: 1rem;
    }

    h2 {
      font-size: 1.5rem;
      font-weight: bold;
    }

    ul.tabs {
      display: flex;
      border-bottom: 1px solid #ccc;
      margin-top: 1.5rem;
```

```

padding-left: 0;
}

ul.tabs li {
margin-right: 0.5rem;
list-style: none;
}

ul.tabs a {
display: inline-block;
padding: 0.5rem 1rem;
text-decoration: none;
font-weight: 600;
color: #3b82f6; /* blue-500 */
border-radius: 0.25rem 0.25rem 0 0;
transition: all 0.2s ease;
}

ul.tabs a:hover {
color: #1e40af; /* blue-800 */
}

ul.tabs a.active {
background-color: white;
color: #1d4ed8; /* blue-700 */
border: 1px solid #ccc;
border-bottom: none;
border-radius: 0.25rem 0.25rem 0 0;
}

.content {
background-color: white;
padding: 1rem;
border: 1px solid #ccc;
border-top: none;
border-radius: 0 0 0.25rem 0.25rem;
margin-top: -1px;
}

[x-cloak] {
display: none !important;
}
</style>
</head>

<body>
<div class="container" x-data="{ tab: 'tab1' }">
  <h2>Tabs</h2>

  <ul class="tabs">
    <li>
      <a href="#"
        :class="{ 'active': tab === 'tab1' }"
        @click.prevent="tab = 'tab1'">
        Tab 1
      </a>
    </li>
    <li>
      <a href="#"

```

```

        :class="{ 'active': tab === 'tab2' }"
        @click.prevent="tab = 'tab2'">
    Tab 2
</a>
</li>
<li>
    <a href="#"
        :class="{ 'active': tab === 'tab3' }"
        @click.prevent="tab = 'tab3'">
    Tab 3
    </a>
</li>
</ul>

<div class="content">
    <div x-show="tab === 'tab1'" x-cloak>
        <strong>Tab I content.</strong> Lorem ipsum dolor sit amet, consectetur
adipisicing elit. Nesciunt sunt, consectetur eos
        quod perferendis mollitia consequuntur natus, porro molestiae qui iusto
deserunt rerum tempore
        voluptatum itaque. Ad, nisi esse cum quidem consequuntur ullam obcaecati.
    </div>

    <div x-show="tab === 'tab2'" x-cloak>
        <strong>Tab II content.</strong> Lorem ipsum dolor sit amet, consectetur
adipisicing elit. Nesciunt sunt, consectetur eos
        quod perferendis mollitia consequuntur natus, porro molestiae qui iusto
deserunt rerum tempore
        voluptatum itaque. Ad, nisi esse cum quidem consequuntur ullam obcaecati.
    </div>

    <div x-show="tab === 'tab3'" x-cloak>
        <strong>Tab III content.</strong> Lorem ipsum dolor sit amet, consectetur
adipisicing elit. Nesciunt sunt, consectetur eos
        quod perferendis mollitia consequuntur natus, porro molestiae qui iusto
deserunt rerum tempore
        voluptatum itaque. Ad, nisi esse cum quidem consequuntur ullam obcaecati.
    </div>
</div>
</div>
</body>
</html>

```

Exemple 4 : Création d'un Menu déroulant

```
<div x-data="{ open: false }">
  <button @click="open = !open">Toggle Menu</button>

  <ul x-show="open" class="menu">
    <li>Accueil</li>
    <li>À propos</li>
    <li>Contact</li>
  </ul>
</div>
```

On va créer un menu qui s'ouvre/ferme avec un effet de transition fluide (CSS pur).

```
<!DOCTYPE html>
<html lang="fr" x-data="{ open: false }">
<head>
  <meta charset="UTF-8">
  <title>Menu déroulant</title>
  <script defer src="https://unpkg.com/alpinejs@3.x.x/dist/cdn.min.js"></script>
  <style>
    .menu {
      overflow: hidden;
      max-height: 0;
      opacity: 0;
      transition: max-height 0.4s ease, opacity 0.3s ease;
    }

    .menu.show {
      max-height: 200px; /* ajuste selon contenu */
      opacity: 1;
    }

    .menu-item {
      padding: 0.5rem;
      border-bottom: 1px solid #ddd;
    }

    .menu-item:last-child {
      border-bottom: none;
    }
  </style>
</head>
<body style="padding: 2rem; font-family: sans-serif;">

  <div>
    <button @click="open = !open">
      {{ Menu }} <span x-text="open ? '▲' : '▼'"></span>
    </button>

    <div :class="open ? 'menu show' : 'menu'" class="menu">
      <div class="menu-item">Accueil</div>
      <div class="menu-item">À propos</div>
      <div class="menu-item">Contact</div>
    </div>
  </div>
```

```
</body>
</html>
```

Explication :

- max-height + opacity sont animés pour un effet fluide
- Alpine gère l'état d'ouverture (open)
- On évite x-show ici pour gérer le CSS manuellement, mais tu peux le coupler avec x-transition pour des animations automatiques

Avec input et formulaire

```
<div x-data="{ name: '' }">
  <input type="text" x-model="name" placeholder="Ton prénom">
  <p>Bonjour, <span x-text="name"></span> !</p>
</div>
```

Ressources utiles

- Site officiel : <https://alpinejs.dev>
- Cheatsheet : <https://alpinejs.dev/cheatsheet>
- Playground : <https://codepen.io/> ou <https://jsfiddle.net/> pour tester en ligne

Exemple 5 : Formulaire de contact avec validation (e-mail & téléphone)

On va faire un formulaire simple avec validation côté client, **sans dépendances**.

```
<!DOCTYPE html>
<html lang="fr" x-data="contactForm()" x-init="init()">
<head>
  <meta charset="UTF-8">
  <title>Formulaire de contact</title>
  <script defer src="https://unpkg.com/alpinejs@3.x.x/dist/cdn.min.js"></script>
  <style>
    body {
      font-family: sans-serif;
      padding: 2rem;
    }

    label {
      display: block;
      margin-bottom: 0.25rem;
      font-weight: bold;
    }

    input, textarea {
      width: 100%;
      padding: 0.5rem;
      margin-bottom: 1rem;
      border: 1px solid #ccc;
      border-radius: 4px;
    }

    .error {
      color: red;
      font-size: 0.9rem;
      margin-top: -0.75rem;
      margin-bottom: 0.75rem;
    }

    button {
      padding: 0.5rem 1rem;
      background-color: #007bff;
      color: white;
      border: none;
      border-radius: 4px;
      cursor: pointer;
    }

    button:hover {
      background-color: #0056b3;
    }
  </style>
</head>
<body>

  <h2><img alt="envelope icon" data-bbox="168 748 185 765"/> Contactez-nous</h2>

  <form @submit.prevent="submit">
    <div>
      <label for="email">Email</label>
      <input type="email" id="email" x-model="email">
      <template x-if="errors.email">
        <div class="error" x-text="errors.email"></div>
      </template>
    </div>

    <div>
      <label for="phone">Téléphone</label>
      <input type="tel" id="phone" x-model="phone" placeholder="ex : 0612345678">
    </div>
  </form>
</body>
```

```

    <template x-if="errors.phone">
      <div class="error" x-text="errors.phone"></div>
    </template>
  </div>

  <div>
    <label for="message">Message</label>
    <textarea id="message" rows="5" x-model="message"></textarea>
    <template x-if="errors.message">
      <div class="error" x-text="errors.message"></div>
    </template>
  </div>

  <button type="submit">Envoyer</button>
</form>

<template x-if="successMessage">
  <p style="color: green; margin-top: 1rem;" x-text="successMessage"></p>
</template>

<script>
  function contactForm() {
    return {
      email: '',
      phone: '',
      message: '',
      errors: {},
      successMessage: '',

      init() {
        // Optionnel : pré-remplissage ou autre init
      },

      validateEmail(email) {
        const regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
        return regex.test(email);
      },

      validatePhone(phone) {
        const regex = /^0[1-9](\d{2}){4}$/; // Format FR : 0X XX XX XX XX
        return regex.test(phone);
      },

      submit() {
        this.errors = {};
        this.successMessage = '';

        if (!this.validateEmail(this.email)) {
          this.errors.email = "Adresse email invalide.";
        }

        if (!this.validatePhone(this.phone)) {
          this.errors.phone = "Numéro de téléphone invalide (format : 06XXXXXXXX).";
        }

        if (!this.message.trim()) {
          this.errors.message = "Le message ne peut pas être vide.";
        }

        if (Object.keys(this.errors).length === 0) {
          // Simuler l'envoi
          this.successMessage = "✅ Message envoyé avec succès !";
          this.email = '';
          this.phone = '';
          this.message = '';
        }
      }
    }
  }
</script>

```

```
</body>  
</html>
```

Résumé des validations :

Champ	Validation
Email	Regex simple pour emails standards
Téléphone	Format français 10 chiffres commençant par 0
Message	Ne doit pas être vide